



USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

Fuzzing Open-Source GPU Hardware with SIMT Program Generation

Zibo Gao, Jie Wang, Qihang Zhou, Lixiao Shan, Junjie Hu, Xiaoqi Jia, and
Zhiqiang Lv, *Institute of Information Engineering, Chinese Academy of Sciences;
School of Cyber Security, University of Chinese Academy of Sciences*

<https://www.usenix.org/conference/usenixsecurity26/presentation/gao-zibo>

This paper is included in the Proceedings of the
35th USENIX Security Symposium.

August 12–14, 2026 • Baltimore, MD, USA

ISBN 978-1-939133-58-8

Open access to the Proceedings of the
35th USENIX Security Symposium
is sponsored by



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

Fuzzing Open-Source GPU Hardware with SIMT Program Generation

Zibo Gao^{†§}, Jie Wang^{†§}, Qihang Zhou^{†§}, Lixiao Shan^{†§}, Junjie Hu^{†§}, Xiaoqi Jia^{†§}, and Zhiqiang Lv^{†§} *

[†]*Institute of Information Engineering, Chinese Academy of Sciences.*

[§]*School of Cyber Security, University of Chinese Academy of Sciences.*

Abstract

GPUs have become critical components in computing systems. In the post-Moore's Law era, the demand for performance is driving increasing GPU microarchitectural complexity, which in turn gives rise to new vulnerabilities.

In this paper, we present the first framework for fuzzing GPU hardware designs at the Register Transfer Level (RTL) to automatically discover vulnerabilities. At the core of FuzzGPU is a novel generator that randomly constructs valid SIMT test programs with complex data and control flow, explicitly exercising the GPU execution model and thereby triggering diverse hardware behaviors. This bridges a key gap: prior processor fuzzers predominantly target CPUs and largely ignore GPU hardware. FuzzGPU further implements a GPU-specific differential testing harness. It tackles microarchitectural non-determinism (e.g., caches and memory timing) and enables trace-driven differential testing against an ISA-level oracle. This technique unlocks vulnerability detection and localization with instruction-level precision. We evaluated FuzzGPU on two real-world open-source GPUs, Vortex and Ventus OpenGPGPU, uncovering 20 previously unknown bugs, including 17 RTL bugs and 3 ISS bugs, 10 of which were assigned CVE identifiers.

1 Introduction

Graphics Processing Units (GPUs) have become essential components in modern computing systems and support a wide range of applications, from artificial intelligence to scientific simulation. In the post-Moore's Law era, the push for higher performance is forcing GPU microarchitectural design to unprecedented levels of complexity. This design complexity, however, elevates the threat of hardware flaws and vulnerabilities [29, 34, 44, 53].

Hardware bugs are *permanent* and *potent* [42]. Once manufactured, a GPU's hard-wired circuitry cannot be rewired, making patches extremely difficult to deploy. Moreover,

flawed hardware may invalidate even formally verified software [21, 50]. This raises a critical question: how can we effectively verify GPU designs to identify hardware bugs and vulnerabilities?

Unfortunately, despite this urgency, finding hardware bugs in GPU designs has received limited attention in existing studies. Previous processor verification methods, including fuzzing-based approaches (e.g., DiveFuzz [10], Cascade [39]) and formal methods (e.g., μ CFI [6]), mainly focus on Central Processing Units (CPUs) and are unable to generate GPU-specific test programs or localize GPU hardware bugs because they do not capture the GPU execution model. Research on GPU testing traditionally focuses on software-level bugs in drivers [18] and utility libraries [54], leaving hardware threats largely unexplored. To the best of our knowledge, no existing work is specifically tailored for fuzzing GPU hardware designs.

Fuzzing GPU hardware designs is non-trivial because GPUs follow the Single-Instruction, Multiple-Thread (SIMT) execution model, where instructions execute in parallel across many threads organized into warps, blocks, and grids. This execution model differs from that of CPUs and introduces *challenges* that existing approaches do not adequately address.

The first challenge is to systematically generate valid and intricate GPU programs to trigger potential bugs. Our analysis reveals that SIMT execution introduces unique control flow, such as thread divergence and synchronization. These features significantly reduce program completion rates and often cause tests to stall prematurely (§3).

The second challenge is the feasibility of detecting GPU hardware bugs. Unlike software that has well-defined exception behaviors such as crashes and memory sanitizer violations, GPU hardware bugs may not manifest as conventional exceptions. Although differential testing between the Device Under Test (DUT) and Golden Reference Models (GRM) has proven practical for hardware bug detection [5, 15, 47, 48, 52], GPUs exhibit several characteristics that complicate traditional differential testing: (i) Multithreaded memory operations can be non-deterministic due to microarchitectural tim-

*Corresponding author: lvzhiqiang@iie.ac.cn

ing (e.g., coherence and caches), while most GPU GRMs are ISA-level and cannot faithfully model these effects [43]. (ii) Inter-warp synchronization introduces ordering dependencies that conventional lockstep checking failed to capture. (iii) GPUs perform out-of-order writeback without reorder buffers (ROBs), unlike CPUs that typically rely on ROBs to provide the precise exceptions required by operating systems. Consequently, the register file state may reflect transient values rather than the architectural state, yielding mismatches when comparing the DUT against the GRM.

Our Contribution. To address these challenges, we present FuzzGPU, the first GPU RTL fuzzing framework that automatically uncovers hardware bugs. FuzzGPU contributes two techniques. (i) We design an ISA-emulation-guided SIMT program generator that produces valid yet diverse parallel tests by explicitly exercising the GPU execution model. The key insight is that *validity is a throughput feature*: the generator avoids stalling the test at short failed attempts and sustains long tests. Concretely, it uses a SIMT-inspired *generation stack* to track divergence and reconvergence with per-thread masks, and a deadlock-free *barrier planner* to impose a consistent ordering of barrier IDs and scopes across warps. (ii) We propose a GPU-aware *instruction-level* differential checking that enables bug detection and instruction-granular localization despite GPU non-determinism. The motivation is that lockstep testing for GPUs differs substantially from that for CPUs. We therefore verify traces against an ISA oracle by (a) re-serializing out-of-order writeback events using instruction IDs, (b) advancing the oracle with a synchronization-aware scheduling policy to avoid barrier-induced ordering ambiguity, and (c) using taint-aware checking to defer ambiguous memory-dependent cases to a memory-consistency validator.

Evaluated on two real-world open-source GPUs (Vortex [41] and OpenGPGPU [27, 28]), FuzzGPU discovers 20 new bugs (17 RTL bugs and 3 ISS bugs, with 10 new CVE IDs). It also outperforms our GPU ports of two state-of-the-art CPU fuzzers, Cascade [39] and DiveFuzz [10], achieving up to $2\times$ higher expression coverage and $28\times$ higher effective fuzzing throughput.

Our key contributions are summarized as follows:

- **Novel SIMT Program Generator.** To the best of our knowledge, this is the first GPU hardware fuzzer that generates valid and intricate SIMT programs to improve instruction throughput and trigger GPU hardware bugs.
- **New GPU Differential Testing Harness.** We propose a GPU-aware differential testing harness for instruction-level bug detection. It remains effective under GPU-specific non-determinism, including relaxed memory behaviors, synchronization, and out-of-order writeback.
- **Evaluation.** We evaluate FuzzGPU on two real-world open-source GPUs (Vortex and OpenGPGPU). We also manually ported two state-of-the-art CPU fuzzers to GPU as baselines for comparison.

2 Background

2.1 GPU Software and Hardware

Modern GPU designs incorporate a multitude of Streaming Multiprocessors (SMs) and High-Bandwidth Memory (HBM), enabling them to effectively accelerate parallel workloads.

GPU Program Structure. A GPU program typically consists of multiple kernels, each launched as a grid of Cooperative Thread Arrays (CTAs) that are dynamically scheduled onto Streaming Multiprocessors (SMs). Each CTA contains multiple concurrently executing and synchronizing warps. A warp is the basic scheduling unit executing a common instruction stream, typically containing 32 threads. A thread is the smallest execution unit, maintaining its own register context and predicate mask, which controls instruction participation for conditional execution and branch-induced divergence.

Thread Divergence. Warps follow the Single-Instruction, Multiple-Thread (SIMT) model, where active threads execute a common instruction stream. A warp is *converged* when all active threads share the same next PC. Branches may cause *divergence* by splitting the thread mask across multiple PCs, followed by *reconvergence* when paths merge. GPUs handle divergence by masking off inactive threads on the current path and using the SIMT stack to record the active masks (TMASK) and reconvergence PCs (RPC) for deferred paths.

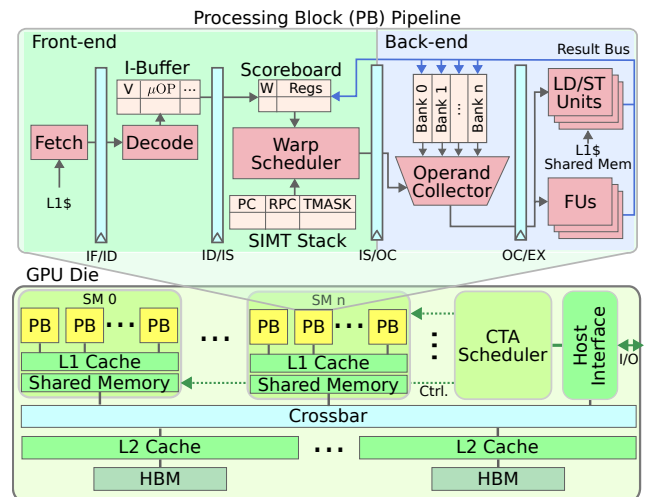


Figure 1: GPU microarchitecture overview.

GPU Microarchitecture. Figure 1 depicts a typical GPU microarchitecture. When a kernel is launched from the host CPU, the host first partitions the kernel into multiple Cooperative Thread Arrays (CTAs) and sends them to the CTA scheduler. The scheduler dynamically assigns received CTAs to SMs. Each SM contains a private L1 cache, shared memory, and a set of processing blocks (PBs) to execute warps in each CTA. PBs decode warps into micro-operations (μ OPs), issued to load/store (LD/ST) or functional units (FUs) that

read operands from register banks, then write back results to registers or memory.

GPUs employ out-of-order writeback to hide LD/ST and FU latencies. A scoreboard is required to ensure correctness by tracking pending register writes and preventing data hazards from dependent instructions. Meanwhile, although warp schedulers select ready warps to hide latency, instructions within a warp remain in-order issued.

2.2 Software and Hardware Fuzzing

Fuzzing has been widely used in software to automatically discover unknown bugs [2, 11, 13, 17, 30, 31, 49]. Recently, fuzzing has been adapted to hardware design verification [3–5, 7, 10, 12, 15, 19, 20, 23, 24, 39, 40, 42, 45–47]. While traditional verification methods like dynamic and formal verification are effective, they are labor-intensive and often suffer from limited coverage [9]. Hardware fuzzing, as an automated testing approach, has proven to be an effective complement to these conventional techniques [7, 37].

State-of-the-art hardware fuzzers generate and mutate random sequences of test instructions to exercise various functionalities and design corner cases. Unlike software fuzzers that generate data streams conforming to specific formats, hardware fuzzers typically produce instruction streams adhering to the Instruction Set Architecture (ISA) to drive hardware state transitions.

In contrast to software checkers that monitor for crashes or memory violations, hardware processors rarely crash: even exceptions are often valid ISA behaviors. While traditional bug detection relies on manual SystemVerilog assertions, recent work demonstrates the superiority of differential testing [5, 15, 47, 48, 52]. These approaches input the same test program into both the DUT and a Golden Reference Model (GRM), comparing their states. State comparison can be performed at program or instruction granularity: program-level checking compares only the final architectural state (e.g., registers and memory), which can miss transient errors that are later overwritten [10], whereas instruction-level checking compares state after each committed instruction, improving localization and reducing false negatives.

3 Motivation and Challenges

Our goal is to effectively trigger and detect GPU hardware bugs via hardware fuzzing. A GPU fuzzer needs to have two key capabilities: (i) generating SIMT test programs to trigger potential bugs; (ii) effectively detecting GPU hardware bugs. In this section, we first analyze the challenges of GPU fuzzing based on these capabilities, then analyze their root causes which guide our design.

SIMT Test Programs Generation. Previous processor fuzzers have primarily targeted CPU hardware. To the best

of our knowledge, no prior work has systematically studied valid GPU program generation for hardware fuzzing.

To understand why applying existing processor fuzzers to GPU designs is insufficient and new techniques are needed, we analyzed the execution results of 5,000 test programs generated by two extended processor fuzzers, Cascade [39] and DiveFuzz [10], that were manually adapted to the Vortex64 GPU [41] design. Specifically, we generate random instruction sequences to construct warps. We then execute them across multiple threads and warps to form CTAs and saturate the maximum hardware-supported concurrency.

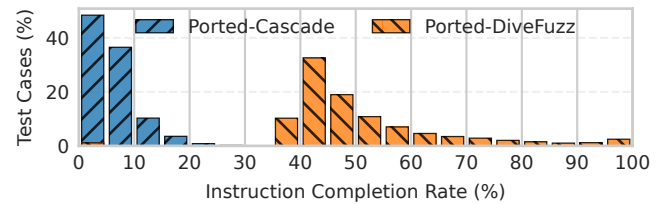


Figure 2: Distribution of instruction completion rates.

Figure 2 shows the histogram of the instruction completion rate, which is the percentage of executed instructions per test case. We evaluate these programs using an ISA simulator (ISS) as oracle. We found that most test programs aborted early, although the generated instructions were legally encoded and the thread context was properly initialized. Therefore, the portion of the valid instructions used for fuzzing is limited. To further analyze the root causes, we run the test under single-thread and single-warp configurations, where the median completion rate recovers to 99% (Cascade) and 98% (DiveFuzz), indicating that multiple threads and warps are the primary source of invalidity. Further inspection reveals that the GPU rejected most tests due to unmodeled GPU-specific behavior (Figure 3). For DiveFuzz, 61% of the rejections are related to SIMT stack errors or Warp-level Matrix Multiply-Accumulate (WMMA) errors, 4% rejections are attributed to barriers. For Cascade, invalid control flow caused by multithreaded non-determinism accounts for a larger fraction (79%). These rejections are permissible under the ISA specification and therefore do not constitute hardware bugs. However, they significantly limit the execution depth of test programs.

Challenge 1: Test Program Validity

New techniques different from CPU fuzzers are required to generate valid and intricate SIMT test programs.

GPU Differential Testing. Another key capability of a fuzzer is to effectively detect vulnerabilities. Unlike software that exposes bugs via memory sanitizers or crashes, GPU hardware bugs may not manifest as conventional exceptions. To accurately locate hardware bugs, instruction-level differential

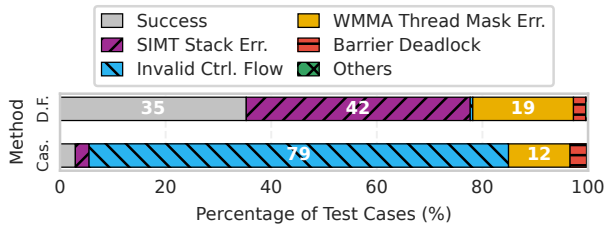


Figure 3: Reasons for rejecting test programs.

testing, which compares execution results between the Device Under Test (DUT) and a Golden Reference Model (GRM), has proven to be a practical approach [5, 15, 47, 48, 52]. However, GPU differential testing is challenging due to GPU-specific characteristics.

Relaxed memory consistency permits memory operations to become visible out of program order across threads unless constrained by synchronization. Consequently, the outcome realized on hardware may depend on microarchitectural timing. However, most GPU GRMs lack microarchitectural fidelity because they simulate only ISA semantics and omit timing-dependent visibility effects (e.g., store buffers, cache-coherence latency). For example, let $R_i(a)$ and $W_i(a)$ denote a load and store to byte a , respectively, issued by warp i . Consider a warp executing a load $R_1(a)$, while other warps execute stores $W_2(a)$ and $W_3(a)$ to the same address. Without synchronization, $R_1(a)$ may legally read the old value or a value written by $W_2(a)$ or $W_3(a)$, depending on the relative propagation timing. In contrast, an ISA-level GRM typically resolves the load using a fixed serialization rule and thus returns a single value for a chosen interleaving, ignoring microarchitectural timing. Thus, the final register state may differ between the DUT and GRM, even if both executions are correct.

Challenge 2: Memory Non-determinism

Non-deterministic GPU memory behaviors can invalidate differential testing.

Moreover, traditional differential testing assumes lockstep execution: the GRM advances in tandem with the DUT, driven by the writeback of the oldest in-flight instruction. However, inter-warp synchronization on GPUs breaks this assumption. The lockstep model inherently fails to capture the dependencies and timing of inter-warp synchronization.

For instance, Figure 4 illustrates the issue in which two warps (W_1 , W_2) synchronize at a shared barrier. Here, a *barrier* denotes a synchronization instruction that stalls each participating warp until all warps in the same barrier group arrive. Suppose warp W_1 in the DUT arrives at a barrier instruction i_1 , causing the GRM to mark W_1 as inactive and wait for it. When W_2 in the DUT subsequently arrives at the matching barrier i_2 , activating W_1 , the GRM faces an *ordering*

ambiguity. If the GRM steps W_1 before observing W_2 's arrival at the barrier, it may simulate W_1 past the barrier while it is still considered stalled, leading to a spurious mismatch.

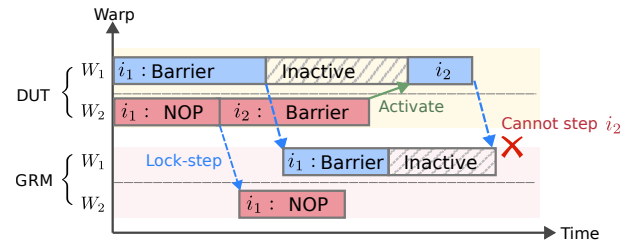


Figure 4: Ordering ambiguity in lockstep testing.

Additionally, GPU cores typically employ out-of-order writeback to improve performance [14, 28, 41]. GPU cores typically do not require the precise exceptions expected by operating systems. Thus, unlike CPUs, they often omit a reorder buffer (ROB) for in-order commit. As a result, the register state observed at any cycle may not be equivalent to the architectural state of the program. Comparing such transient states between the DUT and the GRM can therefore yield spurious mismatches.

Challenge 3: Differential Testing Feasibility

Inter-warp synchronization and out-of-order writeback break conventional lockstep testing.

4 FuzzGPU

4.1 Design Overview

In order to address the aforementioned challenges, we design two novel operating primitives: (i) SIMT program generator that systematically generates valid and intricate SIMT test programs to trigger potential bugs; (ii) differential testing harness that effectively detects GPU hardware bugs with non-deterministic behaviors. Figure 5 illustrates the overall fuzzing loop design of FuzzGPU.

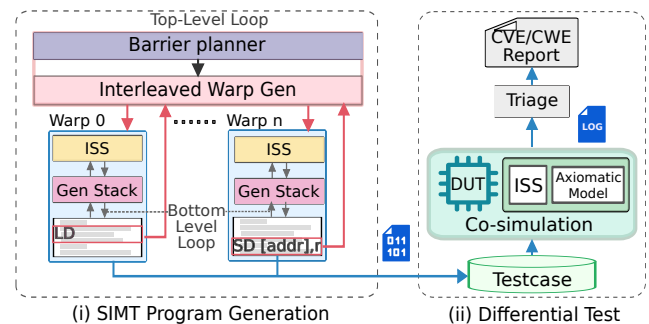


Figure 5: Fuzzing loop overview.

4.2 SIMT Program Generation

As illustrated in the §3, generating valid SIMT programs is non-trivial due to the unique GPU execution model. In response, we decompose the generation into a novel top-down process, as shown in Figure 5. A top-level loop plans the overall program structure, while a bottom-level loop generates the instruction stream for each warp. These loops handle inter-warp and intra-warp interference respectively, ensuring the resulting program follows the GPU execution model while keeping it intricate.

4.2.1 Target Program Structure

The two-level generator loop constructs the overall structure of the GPU program, including multiple CTAs, warps, and basic blocks.

Instructions. The target program follows the GPU hierarchy: it contains CTAs, warps, and a per-warp instruction stream. We categorize instructions as control-flow transfer (e.g., branches, unconditional jumps, reconvergence, and join) or non-control-flow. For a control-flow instruction, if the next PC falls through to the immediate successor instruction, it is *not-taken*. Otherwise, the instruction is classified as *taken*.

Basic Blocks. A per-warp instruction stream can be viewed as a sequence of basic blocks. A basic block is a straight-line instruction sequence with a single entry and a single exit, which ends at a control-flow transfer instruction. For each block, the generator allocates a disjoint memory region and assigns an entry address. The block's instruction sequence is randomly sampled subject to data-flow and control-flow constraints (§4.2.2). Blocks are connected by control-flow transfer instructions, forming a Control-Flow Graph (CFG).

4.2.2 Per-Warp Generation Loop

At the bottom level, the per-warp loop constructs basic blocks for each warp. A warp executes a single instruction stream as long as its threads are converged. Thus, we use one Generation Program Counter (GPC) to determine the addresses at which instructions are generated. However, after divergence, threads may follow different control-flow paths, implying multiple feasible PCs and potentially exponential path growth with branch depth. Naively emitting disjoint code for both directions of every branch wastes instruction budget on dead code, whereas emitting only one direction can prevent threads from reconverging and yield a subset of threads permanently excluded. Therefore, generation must emit both paths while ensuring that they eventually merge at a reconvergence point.

Generation Stack. Inspired by the hardware SIMT stack, we design a *generation stack* to track divergence and reconvergence during per-warp code emission. Guided by an ISA Simulator (ISS), the generator resolves branch outcomes and emits both directions of divergent branches while maintaining a single GPC. Each stack entry stores the branch PC,

the current active thread mask (TMASK), the not-taken target PC and mask (NT), and a reconvergence flag R . When sampling a branch, an ISS predicts per-thread next masks. Divergence occurs if both taken and not-taken paths are active. The generator then pushes an entry and generates the taken path depth-first. Upon sampling a reconvergence instruction, it jumps to the pending not-taken target from the stack top, generates that path, and sets R . When the not-taken path reaches the reconvergence point again with R already set, the entry is popped, and generation resumes after reconvergence. This procedure yields a linearized order in which all threads share the same GPC at each step, corresponding to a valid interleaving of threads.

Figure 6 shows an example of the generation stack. Initial execution diverges at A. Prioritizing the higher active thread count, the generator follows branch B (threads 0011), pushing A's metadata onto the stack. Generation proceeds along path ① until the join point G. Seeing $R = 0$ at the top of the stack (i.e., an alternative path is still pending), the generator redirects the GPC to the not-taken path C (threads 1100), which then diverges into paths ② and ③. This recursive process continues until the GPC returns to G with $R = 1$ (signifying full reconvergence), prompting a stack pop and resumed generation after G.

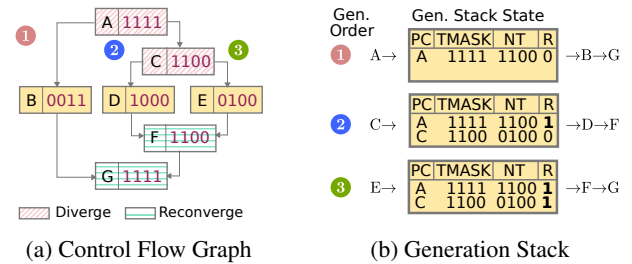


Figure 6: Generation stack examples.

The linear order induced by the generation stack has several desirable properties: (i) it enables using the ISS during generation to obtain runtime context and resolve branch outcomes, allowing the procedure to bootstrap; (ii) it avoids maintaining a separate PC for every feasible path, mitigating path explosion; and (iii) it guarantees the absence of dead code in the linearized order, providing a sound program order for the barrier planner and memory-consistency checking.

4.2.3 Top-level Generation Loop

The top-level generation loop coordinates multi-warp execution by: (i) placing synchronization barriers across scopes to prevent deadlocks; and (ii) interleaving the generation of memory operations to track potential cross-thread and cross-warp conflicting accesses.

Barrier Planner. GPU warps coordinate execution via synchronization barriers. However, improper barrier placement

can lead to deadlocks, where warps wait indefinitely. Given a set of named barrier operations $B = \{b_1, b_2, \dots, b_k\}$, each distinguished by an ID and a scope $(b) \subseteq G$ (where G is the set of all warps), a barrier is released only after all participants in its scope have arrived at the barrier with the matching ID.

We first analyze the deadlock conditions. Figure 7 illustrates warp execution under different barrier placements. Figure 7a shows a deadlock where warp 0 waits for warp 1 at barrier B_1 , while warp 1 waits for warp 0 at B_2 , stalling both. Conversely, Figure 7b shows a valid placement where both warps reach the same barrier B and proceed without deadlock.

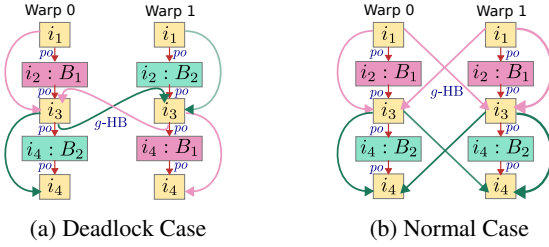


Figure 7: Barrier examples.

Then, we formally model the deadlock conditions. Let I_w be the instruction sequence of warp w , and let po_w denote the program order relation, where $i_1 \xrightarrow{po} i_2$ if and only if i_1 is encountered before i_2 in the linearized emission order for warp w . Since po_w is derived from the linearized emission order induced by the generation stack (§4.2.2), the generator avoids emitting unreachable dead code in this order: each emitted instruction is eventually encountered by the warp under a well-defined thread mask. We define the group-barrier happens-before relation ($g\text{-HB}$) for a barrier b in group g :

$$\forall w_1, w_2 \in \text{scope}(b),$$

$$\forall o_1 \in \text{Prec}_b(w_1), \forall o_2 \in \text{Succ}_b(w_2) : o_1 \xrightarrow{g\text{-HB}} o_2 \quad (1)$$

where $\text{Prec}_b(w) = \{o \in I_w \mid o \xrightarrow{po_w} b\}$ and $\text{Succ}_b(w) = \{o \in I_w \mid b \xrightarrow{po_w} o\}$ describe the sets of instructions that precede and succeed barrier b in warp w , respectively.

Deadlocks fundamentally arise from cycles in the graph formed by $g\text{-HB} \cup po$. Such a cycle signifies a circular dependency where every instruction in the loop must await the completion of its predecessor, thereby halting progress indefinitely. A valid barrier placement must ensure that $g\text{-HB}$ respects the program order po_w , thereby preventing cycles.

We propose a barrier planner that pre-constructs a global barrier program order. To avoid interleaving-induced cycles, it enforces a consistent barrier-ID sequence across warps. The planner samples a globally unique sequence B of records (id, scope) , and uses B as a skeleton into which it intersperses random non-barrier instructions. Scopes are chosen stochastically from those supported by the target: global and CTA scopes on Vortex, and CTA scope on OpenGPGPU. During

per-warp generation, the generator maintains a cursor $c(w)$ into B (initialized to 0). When a barrier opcode is sampled, warp w consults $B[c(w)]$ and emits the barrier iff $w \in \text{scope}$. Once all warps in the scope have emitted it, the record is removed from B to prevent duplication. Finally, $c(w)$ advances to the next record. A warp concludes generation only when $c(w)$ reaches the end of B . Figure 8 shows a random program skeleton generated by the barrier planner for 5 warps. A proof of the barrier planner's deadlock freedom is provided in Appendix A.

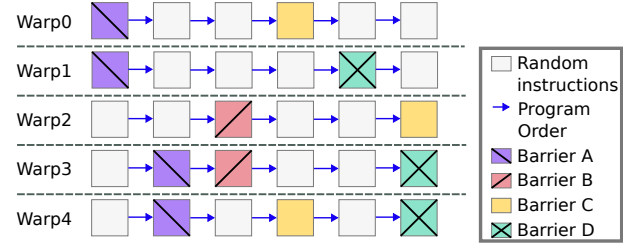


Figure 8: Barrier planner example: barriers A–D have 4, 2, 3, and 3 arriving warps, respectively.

Interleaved Warp Generation. GPU loads and stores provide a mechanism for inter-warp communication. However, under weak or relaxed memory consistency, a load may observe different values depending on the timing of write propagation and arbitration. Because an ISA-level Golden Reference Model (GRM) typically resolves each load deterministically, naive differential testing can misclassify such correct-but-different DUT behaviors as bugs. We therefore track memory accesses online and classify potentially conflicting loads for later memory-model validation.

We implement an interleaved warp generation loop. Each warp emits instructions in the intra-warp linear order induced by the generation stack. When it reaches a memory instruction, the generator records its effective address and operands, pauses the current warp generation, and switches to another warp. After all per-warp generations have paused, a global pass identifies conflicts and taints the corresponding loads. We define a load as conflicting if it reads an address that another thread has written or will write.

To increase the chance of conflicts, we implement a load-store replay mechanism that keeps a history of effective addresses from the last k memory instructions. The generator induces an address collision by sampling a previously observed address A and choosing the immediate $imm = A - V$ relative to the current base register V . If the attempt is invalid, it resamples and retries.

Taint Registers and Memory. The conflicting loads above are a concrete source of non-determinism. Under weak or relaxed consistency, the write observed by a load is not uniquely determined by program order and may vary depending on the microarchitectural timing. As a result, the DUT may legitimately differ from an ISA-level GRM. We capture this and

other sources of divergence using a *taint* abstraction. We taint a register or memory address when its hardware value may legally differ from the ISS due to unmodeled microarchitectural timing and cross-thread memory conflicts.

During differential testing, we send tainted loads to the memory validator and copy the DUT load results into the GRM context. We treat these tainted results as *don't-care* for ISA-level comparison to prevent cascading mismatches, while the correctness of the tainted memory behaviors is validated separately by the memory validator.

Tainting also helps filter known hardware bugs. Ideally, bugs should be fixed immediately after discovery. However, in practice, remediation often takes substantial time. After root-cause analysis, we add implicated registers, addresses, and opcodes as taint-based generation constraints, so the fuzzer avoids synthesizing instructions that persistently hinder testing or excludes their tainted results from differential testing.

Additionally, if a load/store address depends on tainted registers, non-determinism may produce unaligned addresses. The generator therefore resamples operands to keep addresses legal. For branches with tainted operands that may desynchronize control flow, we insert an unconditional jump from the not-taken to the taken path. If no untainted operand sources remain, we refresh a random subset of registers with randomized immediate constants.

Taint Propagation. The generator tracks taint online as software metadata, requiring no GPU RTL changes. For each candidate instruction, it uses the ISS to emulate execution, recover operands and effective addresses, and update per-warp register tags and per-address memory tags. Taint propagates along data flows via the following rules: (i) tainted source registers taint destination registers; (ii) stores with tainted data or address operands taint the written addresses; (iii) loads with tainted addresses taint destination registers.

4.2.4 Random Instruction Selection

Within each basic block, the generator stochastically populates the instruction stream to induce diverse behaviors. For each instruction, it samples an opcode from a categorical distribution whose parameters are drawn i.i.d. from a uniform distribution, and then selects operands (registers, immediates) and control bits according to the ISA. Control-flow opcodes can induce thread divergence and are therefore generated by the bottom-level loop (§4.2.2).

For memory opcodes, the generator randomly assigns access sizes and effective addresses. To avoid uninitialized reads, it consults a randomized allocator that maintains an ordered list of allocated regions: each load/store computes its effective address and performs a binary search over this list. If the address is not covered, the allocator creates a new region, initializes it with random values, and registers it. For stores, we additionally reject candidates overlapping code or read-only segments. Invalid addresses are resampled until a threshold is

reached, after which a new opcode is selected.

While most instructions allow flexible operand choices, Warp-level Matrix Multiply-Accumulate (WMMA) and control/status-register instructions impose strict constraints. We enforce these constraints (e.g., on the active thread count or rounding mode) through rejection sampling, repeatedly resampling operands until a valid configuration is obtained.

4.3 Differential Testing Harness

To address the challenges discussed in §3, we propose a differential testing framework that: (i) reorders writeback traces using instruction IDs to serialize out-of-order events; and (ii) constructs microarchitectural event graphs to verify memory consistency using axioms.

Untainted Operations. We verify instructions whose results are not tainted (primarily non-memory instructions) by comparing per-instruction writeback traces between the DUT and GRM. Each writeback event records the PC, thread mask, destination register index, and written value.

Because GPU instructions may complete and write back out of order, conventional lockstep differential testing becomes invalid (§3, Challenge 3). We therefore reorder writeback events using the globally unique instruction IDs assigned at the issue stage and propagated to writeback, thereby recovering a deterministic order consistent with per-warp issue order.

The software maintains a per-warp completion queue (FIFO) to track instruction status, recording the instruction ID, writeback status (pending/completed), and the associated trace, as shown in Figure 9. We instrument the design using DPI-C to capture issue and writeback events. When an instruction is issued, its ID is enqueued with a pending status. Upon writeback, we locate the corresponding record using the instruction ID, mark it as completed, and populate it with the hardware writeback data, including register index and value.

The testing harness periodically polls the head of the completion queue. When the head instruction is marked completed, the GRM advances the corresponding warp by one instruction to generate a reference trace, which is then compared against the hardware trace. This process serializes out-of-order writeback events.

However, an arbitrary warp stepping order can violate inter-warp synchronization assumptions and introduce spurious mismatches (§3, Challenge 3). To preserve synchronization semantics, we propose a heuristic scheduling policy that maintains a counter of in-flight barrier instructions per warp, as shown in Figure 9. The counter is incremented when a barrier is issued and decremented when that barrier is dequeued from the completion queue. When multiple queues are ready, a scheduler prioritizes the warp with the non-zero counter before stepping any instructions, which ensures that all warps involved in a barrier reach it before any proceed past it.

Tainted Operations. Relaxed GPU memory consistency permits multiple legal read-from (*rf*) choices, and taint in Fuz-

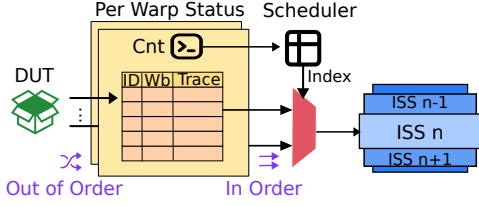


Figure 9: Per-warp completion queue with inter-warp synchronization scheduler.

zGPU covers only this read-from selection uncertainty. For tainted loads, we defer checking to a memory consistency validator that uses an axiomatic model as the reference. After validating the load, we synchronize the data-race load's writeback results to the ISS to validate subsequent operations.

Axiomatic memory models characterize correctness via relations over memory primitives (reads, writes, and fences): an execution trace is valid iff it satisfies the axioms. Open-source GPU models largely share a common relational core, while vendors often add proprietary rules. We present our validator using a widely recognized weak memory model [32].

Table 1: Memory Consistency Axioms

Axiom 1(SC-per-Location).

$\text{acyclic}(\text{morally_strong} \cap (\text{rf} \cup \text{co} \cup \text{fr}) \cup \text{po_loc})$

Axiom 2(Main Model).

$\text{acyclic}(\text{co} \cup \text{rfe} \cup \text{fr} \cup \text{ppo})$

Axiom 3(Atomic).

$\text{empty}(((\text{morally_strong} \cap \text{fr}); (\text{morally_strong} \cap \text{co})) \cap \text{rmw})$

Intuitively, the validator models an execution as a memory-event graph and rejects forbidden cycles. Axiom 1 enforces per-location coherence by requiring a consistent order for same-address accesses. Axiom 2 extends ordering across addresses through dependencies, fences, and cross-thread communication. Axiom 3 ensures RMW atomicity by forbidding any visible write between its read and write parts.

We summarize the relations in the axiomatic model below. $a \xrightarrow{\text{po}} b$ denotes that a precedes b in program order. po_loc restricts po to instructions accessing the same memory address. Two instructions are *morally_strong* if they are ordered by po , or if both are strong instructions with the same scope and overlapping addresses. co is a partial order over globally visible stores, only morally strong stores participate and are comparable. $a \xrightarrow{\text{co}} b$ holds if a and b store to the same location and a becomes globally visible before b . The read-from relation rf links loads to their source stores: $a \xrightarrow{\text{rf}} b$ iff load b reads the value written by store a . We derive the from-read relation fr from co and rf . $a \xrightarrow{\text{fr}} b$ holds if load a reads from a store c such that $c \xrightarrow{\text{co}} b$. Intuitively, fr captures that once a read

observes a store c , it must be ordered before any later store b to the same location in co . Additionally, ppo (preserved program order) captures the subset of program-order edges between memory operations that the ISA requires to be preserved (e.g., due to dependencies and fences), allowing others to reorder. Relations spanning threads are termed external and are denoted by the suffix 'e'. For example, rfe denotes the rf edges whose endpoints are in different threads or warps.

For memory-model validation, litmus tests are widely used [1, 26, 33]. However, these tests are inherently directed: they target preselected relation patterns (e.g., specific cycles) rather than checking arbitrary observed traces against the model. For instruction-level differential testing, we use DPI-C instrumentation to capture memory events from the GPU. Specifically, we hook the L1 cache response handshake. For each read, we record its physical address and the returned data from the ISS. We then query the generator's store map with the resulting (address, data) pair to uniquely identify the source store and establish rf . The store map is built during the generation phase. We employ rejection sampling to assign unique values to stores targeting the same physical address. This eliminates ambiguity. The generator also emits dummy write events ahead of all other microarchitectural events to represent initial memory values.

Detecting write events necessitates tracking L1 and L2 cache microarchitectural states. Because co is a partial order, writes that are not *morally_strong* do not participate in co . For each scope, we maintain a last-writer map $W[\text{scope}][\text{addr}]$ that records the metadata of the most recent write to address addr (e.g., instruction ID and thread mask). For each write, we check the last writer record $W[\text{scope}][\text{addr}]$. If present, we assert a coherence order co from the last to the current writer, then update $W[\text{scope}][\text{addr}]$.

Using the detected rf and co relations, we derive fr and rfe . We construct the memory event graph by combining per-warp sequential order relations (po , po_loc) with ISA-defined ppo rules, and verify the axioms in Table 1. Detected cycles trigger a traceback of the involved events to localize potential memory consistency bugs.

Completeness and Soundness Analysis. The validator is complete with respect to the implemented axiomatic memory model: after FuzzGPU reconstructs the relevant po , rf , co , fr , and ppo relations, any violation of the encoded axioms appears as a failed graph check. Meanwhile, its soundness is practical rather than absolute. False positives can still arise if the target's intended memory model differs from the encoded axioms, or if non-memory known bugs corrupt the graph events. We mitigate false positives by tainting known mismatching states.

5 Evaluation

In this section, we evaluate FuzzGPU's effectiveness from the perspectives of program metrics (§5.1), fuzzing performance

(§5.2), coverage (§5.3), and bug discovery (§5.5).

Experiment Setup. We conducted our experiments on an Ubuntu 24.04 system featuring an Intel(R) Xeon(R) 6982P-C 192-core vCPU and 768GB of RAM. For RTL simulation, we utilized the widely adopted Verilator 5.034.

We tested the latest revisions of the open-source Vortex-64bit and OpenGPGPU (ventus-env), corresponding to commit IDs 2189194 and 7e97907, respectively. We retained the default configuration parameters for all designs. For each DUT, we conducted a 20-core-hour fuzzing campaign using 10 distinct initial seeds. The maximum instruction length per test program was uniformly sampled from 100 to 10,000.

The instruction pool covers all hardware-supported ISA extensions, including integer (RV32I/RV64I), multiplication/division (RV32M/RV64M), atomics (RV32A/RV64A), floating-point (RV32F/RV64F, RV32D/RV64D), vector (Ventus-RVV), and Vortex-customized instructions.

Baselines. To the best of our knowledge, no publicly available GPU hardware fuzzers exist for comparison. Since existing processor fuzzers primarily target CPUs and are inapplicable to GPUs, we established baselines by adapting two state-of-the-art CPU fuzzers, DiveFuzz [10] and Cascade [39]. We re-implemented them to support GPU ISAs, generating per-warp random instruction sequences augmented with thread initialization to form GPU programs. Our baseline porting follows a minimality principle: we only replace ISA encoding and toolchain backends and add the minimum required initialization to launch GPU warps. These adaptations, named Ported-DiveFuzz and Ported-Cascade, utilize our differential testing framework for bug detection.

5.1 Program Metrics

Instruction Completion Rate. Our motivation analysis highlighted the impact of GPU-specific execution models on instruction completion. We assessed the ratio of executed instruction PCs to total generated instructions in FuzzGPU by running 5,000 test programs on an ISS oracle.

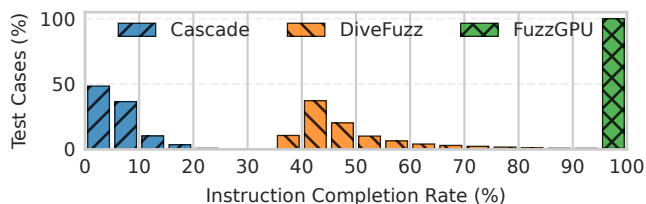


Figure 10: Completion rate of different fuzzers.

Figure 10 shows that FuzzGPU consistently outperforms the baselines. It achieves a median instruction completion rate of 100% and a mean of 99.8%. DiveFuzz and Cascade exhibit low completion rates (medians of 5.0% and 45.4%, respectively) due to their inability to handle GPU execution models,

as triaged in Figure 3. DiveFuzz surpasses Cascade because its seed construction employs dedicated counter registers and constant comparison values for conditional branches, thereby reducing the likelihood of branch misconstruction.

Memory Event Statistics. Table 2 summarizes the scale of memory-relation edges induced by executing each Vortex64 program generated during 20 core-hours of FuzzGPU. We expand both scalar and vector load/store operations at byte granularity. All initial memory reads are represented as *r_{fi}* edges from the dummy initial value write events. The results show that FuzzGPU exercises a large number of memory events, including cross-thread relations such as *coe* and *r_{fe}*, thereby providing rich scenarios for memory-consistency checking.

Table 2: Memory relation edge statistics per test: *co* coherence order, *r_f* read-from, *f_r* from-read, and *ppo* preserved program order. Suffix ‘*e*’ marks external cross-thread edges, and *r_{fi}* reads from initial writes.

Relation	Min	Q1	Median	Q3	Max
<i>co</i>	0	96	312	636	1645
<i>coe</i>	0	24	284	624	1644
<i>r_f</i>	0	9524	20124	30335	44308
<i>r_{fe}</i>	0	140	678	1954	9203
<i>r_{fi}</i>	0	9148	18847	27362	36633
<i>f_r</i>	0	0	160	960	13453
<i>ppo</i>	0	1034	2858	5160	20587

5.2 Fuzzing Performance

To quantify the impact of increased instruction length on throughput, we measured the generation throughput, defined as the ratio of generated fuzzing instructions to the total generation time, while varying the number of generated instructions. For each configuration, we ran Vortex for 0.4 core hours and reported the geometric mean.

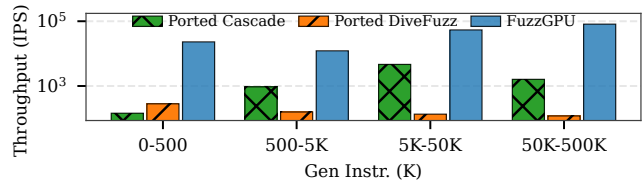


Figure 11: Generation throughput (y-axis is in log scale).

Figure 11 shows that as the number of generated instructions increases, FuzzGPU’s throughput initially rises before plateauing. Overall, FuzzGPU achieves significantly higher generation throughput than the baseline generators. In the range of 5,000 to 50,000 instructions, FuzzGPU maintains a throughput of approximately 53,970 instructions/second, which is 11× that of Ported Cascade and 399× that of Ported

DiveFuzz. DiveFuzz exhibits lower throughput because its core algorithm relies on dynamic guidance from an ISS. As previously noted, its lack of a GPU execution model often results in programs with low instruction completion rates (§3). Consequently, many instructions trigger exceptions during simulation, causing them to be discarded and state rollbacks. In contrast, Cascade only invokes the ISS at the final stage of program generation, resulting in higher throughput than DiveFuzz.

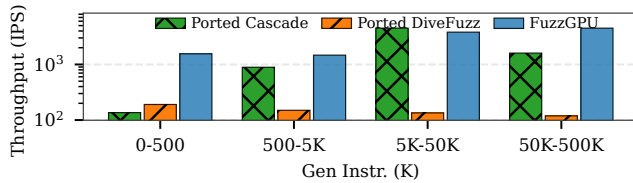


Figure 12: Effective throughput (y-axis is in log scale).

To evaluate effective instruction throughput, we measured the number of actually executed fuzzing instructions per unit of generation and simulation time. These measurements were conducted on Vortex, comparing FuzzGPU against various baselines.

Figure 12 shows that FuzzGPU’s effective throughput initially increases before plateauing within the tested range. Overall, FuzzGPU achieves higher effective throughput than Cascade and DiveFuzz. Cascade’s effective throughput follows a pattern of initial increase followed by a decline, consistent with the previous observations on CPUs [39]. In the range of 5,000 to 50,000 instructions, FuzzGPU maintains an effective throughput of approximately 3,815 instructions/second, which is $28\times$ higher than Ported DiveFuzz and comparable to Ported Cascade.

To further understand simulation performance under differential testing, we measured the geometric mean of simulation times across different instruction generation lengths.

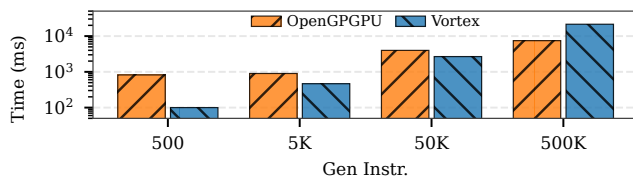


Figure 13: Performance of simulation (differential testing).

At an instruction count of 500,000, the simulation throughput of Vortex64 is approximately 21,576 instructions per second, whereas the average simulation throughput of OpenGPGPU is about 7,654 instructions per second, as shown in Figure 13. This discrepancy is likely attributable to differences in code structure and design complexity.

We measured the runtime breakdown of the generator, simulation, non-memory operation checking, and memory check-

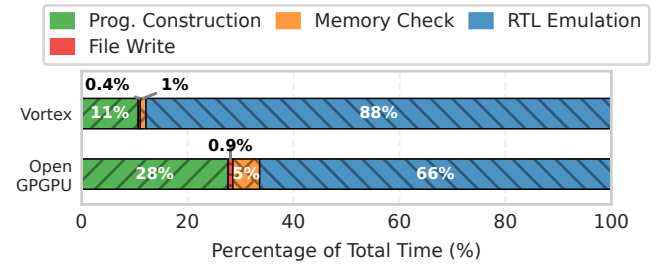


Figure 14: Breakdown of fuzzing time per test.

ing for each GPU test during the 20-core-hour fuzzing campaign. Figure 14 illustrates the breakdown of fuzzing time, specifically the proportions of generation, binary writing, and simulation (differential testing) time relative to the total duration. We found that generation accounts for only a small portion of the total time, approximately 28% on OpenGPGPU, whereas simulation dominates at 66%, and memory checking accounts for 0.9%.

5.3 Coverage Evaluation

We conducted a 12-core-hour fuzzing campaign, utilizing Verilator to collect coverage metrics supported by the DUT.

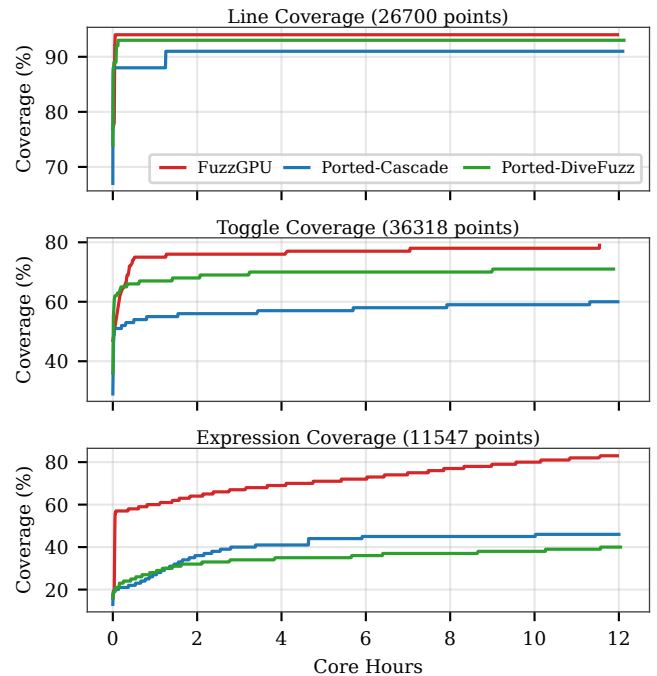


Figure 15: Line, toggle, and expression coverage comparison.

Figure 15 presents the coverage achieved by different fuzzers on the OpenGPGPU design. FuzzGPU consistently

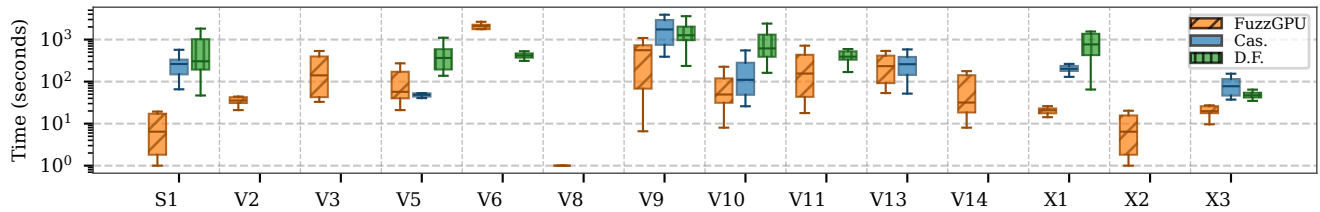


Figure 16: Time to bug discovery across different fuzzers and GPU designs. Detection of V1 and V12 involves time-consuming waveform analysis. V4, V7, P1 and S2 are patched to avoid crashes, as detailed in Appendix B. Time is measured in core-seconds.

achieves higher coverage across all three metrics. Line coverage is relatively the easiest to saturate. A simple test often covers the majority of code lines. FuzzGPU achieves 94% line coverage. In contrast, toggle and expression coverage require more diverse tests to trigger different signal transitions and expression evaluations. FuzzGPU achieves approximately 80% coverage for both toggle and expression metrics. Notably, for expression coverage which is a more challenging metric, FuzzGPU achieves $2\times$ that of Ported Cascade.

5.4 Ablation Study

To validate the effectiveness of FuzzGPU’s moving parts, we conducted an ablation study by disabling: (i) the generation stack (which inherently disables simulation guidance), (ii) the barrier planner, and (iii) taint propagation in interleaved warp generation. We profiled the execution of 5,000 test programs, each generated with a fixed length of 10,000 instructions on the OpenGPGPU ISS.

Impact on Program Metrics. Removing the generation stack reduces the median instruction completion rate from 97.3% to 5.6%. It also removes divergence: the median divergent-branch ratio drops from 19.1% to 0%, and the median SIMT stack depth decreases from 8 to 0. Without the barrier planner (i.e., with random barrier generation), the median number of completed barriers decreases from 35 to 15 per test. The median instruction completion rate also drops to 88.6%, suggesting that the barrier planner mitigates deadlocks. Disabling taint propagation during interleaved warp generation reduces the completion rate to 17.1% and lowers the median SIMT stack depth from 8 to 6. This indicates that taint propagation is important for high completion rates under non-determinism.

Impact on Coverage. We measured final expression coverage on OpenGPGPU across 100 random seeds under the ablations above. The vanilla configuration reaches 59%. Disabling the generation stack reduces coverage to 37%. Removing taint propagation in interleaved warp generation reduces it to 41%. Excluding the barrier planner reduces it to 58%. These results show that each component contributes to higher coverage.

5.5 Bug Discovery Efficiency

To evaluate the effectiveness of differential testing in bug discovery, we summarized the bugs identified across various designs during a 20-core-hour fuzzing campaign. In total, we discovered 20 previously unknown bugs: 14 in OpenGPGPU, 3 in Vortex, and 3 in the ISS, validating the effectiveness of FuzzGPU’s bug detection capabilities. Table 3 provides a summary of these bugs. Compared to the native regression test suites of Vortex and OpenGPGPU, FuzzGPU uncovered 17 previously unknown RTL bugs and 3 ISS bugs.

Time to Bug Discovery. We compared the time required to discover bugs across different fuzzers.

Figure 16 illustrates the time to first detection for each bug during a 20-core-hour run, repeated 10 times with distinct random seeds. Notably, Ported Cascade and Ported DiveFuzz were used only for instruction generation. They cannot independently detect GPU bugs. We therefore use the proposed differential testing framework as the sole bug oracle, using an identical checking configuration across all generators. This comparison isolates how well the generated tests trigger bugs.

The results indicate that FuzzGPU discovers bugs within 1,000 core-seconds in most cases and, overall, achieves a shorter time to bug discovery than the baselines. FuzzGPU successfully triggered and identified 14 bugs in Figure 16, whereas the programs generated by Ported Cascade and DiveFuzz triggered 7 and 8 bugs, respectively, under our differential testing framework. This demonstrates that FuzzGPU’s generation techniques effectively synthesize programs that expose hardware bugs.

5.6 Bug Analysis

Memory Bugs. FuzzGPU identified a private memory address calculation bug (V1). When active threads are fewer than the hardware capacity, the GPU miscalculates physical addresses and triggers out-of-bounds accesses. FuzzGPU also detected a flaw in atomic memory operations (V2): instructions such as `amoadd` and `amoswap` produce incorrect arithmetic or logical results. Finally, the hardware ignores the `nfield` parameter in vector load/store instructions (V3). It defaults to one field, truncating operations such as `vlsseg2e32`.

Table 3: Summary of discovered bugs.

Design	Bug Description	CVE/#Issue/#PR	CWE
OpenGPGPU	(V1) Incorrect private memory address calculation.	CVE-2025-70015	CWE-119
	(V2) Atomic instructions yield incorrect results.	PR #105	CWE-682
	(V3) Vector load/store ignores <code>nfield</code> .	CVE-2025-70011	CWE-682
	(V4) FP ops corrupt integer registers.	Issue #97	CWE-682
	(V5) Vector compare reads vector instead of scalar.	CVE-2025-70009	CWE-682
	(V6) The MISA register was improperly initialized.	CVE-2025-70014	CWE-665
	(V7) Thread ID CSR yields invalid values when only a portion of threads are active.	CVE-2025-70012	CWE-665
	(V8) Inactive threads perform write-back.	CVE-2025-70007	CWE-787
	(V9) <code>vfmax</code> returns hardcoded zero on NaN.	CVE-2025-70010	CWE-682
	(V10) Multiply-high returns the low 32 bits.	CVE-2025-70008	CWE-682
	(V11) Spurious register write-back in tensor ops.	CVE-2025-70016	CWE-119
	(V12) I-Cache deadlock on specific sequences.	PR #108	CWE-833
	(V13) No write-back for certain <code>vmv.x.s</code> , causing SM hangs.	Issue #98	CWE-833
	(V14) Vector mask-register logical operations yield incorrect results.	PR #106	CWE-682
Vortex 64	(X1) Scoreboard releases the freed entry.	PR #356	CWE-672
	(X2) Conditional zero ops show value mismatches against the ISS.	PR #358	CWE-682
	(X3) Tensor ops corrupt registers under BF16 and FP16 output formats.	PR #359	CWE-682
OpenGPGPU-Spike	(P1) Improper sign-extend for 32-bit results, misleading conditional branches.	CVE-2025-70013	CWE-682
Vortex-Simx	(S1) <code>sra</code> is implemented as a logical right shift.	PR #320	CWE-682
	(S2) <code>jalc</code> ignores LSB clearing.	PR #339	CWE-682

Register Bugs. FuzzGPU discovered a register corruption bug (V4) where floating-point results are incorrectly written to integer registers sharing the same index when register extension prefixes are used, thereby corrupting subsequent integer operations. FuzzGPU also identified an operand fetch error (V5) in vector comparison instructions (`vmsgt.vx`, `vmsgtu.vx`), where the hardware erroneously reads from vector registers instead of the specified scalar registers. Moreover, FuzzGPU detected a use-after-free vulnerability in the Vortex scoreboard logic (X1), occurring when the hardware attempts to release registers that are not currently in use.

Configuration and State Management Bugs. FuzzGPU uncovered multiple configuration and state management issues. First, the MISA register in OpenGPGPU, which declares ISA extension support, is improperly initialized, causing undefined behaviors for valid instructions (V6). Second, when executing with a partial thread mask, the CSR register tracking the thread ID fails to update, leading to incorrect ID retrieval and subsequent errors in thread-dependent computations and memory accesses (V7). Lastly, OpenGPGPU hardware erroneously performs write-backs for inactive threads when the active count is below the maximum (V8). While this poses a risk of context corruption, the current OpenCL stack does not utilize this configuration.

Calculation Bugs. FuzzGPU detected several arithmetic errors. In OpenGPGPU, the `vfmax` instruction violates the RISC-V standard by returning a hardcoded zero instead of the valid operand when the other operand is NaN (V9). Additionally, Multiply High instructions (`mulhu`, `vmulhu`, `mulhsu`,

`vmulhsu`) incorrectly yield the lower 32 bits of the product rather than the upper 32 bits (V10). Moreover, the vector mask-register logical operations such as `vmand.mm` yield incorrect results that are inconsistent with Spike (V14). In Vortex, the generated conditional zero operations produce flags that mismatch the ISS, causing erroneous behavior in subsequent control-flow instructions that rely on this flag (X2). The root cause likely stems from inconsistent instruction decoding between the RTL and the ISS.

Tensor Core Bugs. In Vortex, tensor multiply-accumulate instructions corrupt the architectural state (X3) because FP16 and BF16 accumulator outputs are incorrectly generated. In OpenGPGPU, FuzzGPU detected erroneous register write-backs caused by tensor core instructions that executed without triggering an illegal-instruction fault, even though the current open-source release (V11) does not include a tensor-core unit.

Microarchitectural Deadlock Bugs. OpenGPGPU exhibits a microarchitectural deadlock where specific instruction sequences cause the instruction cache to hang the Streaming Multiprocessor (SM), halting all subsequent instruction retirement (V12). Additionally, the `vmv.x.s` instruction hangs the entire SM, halting subsequent write-backs (V13).

Simulator Bugs. FuzzGPU also uncovered a discrepancy in the Vortex ISS, where the arithmetic right shift (`sra`) is incorrectly implemented as a logical right shift (S1). Moreover, Vortex fails to clear the least significant bit (LSB) of PC during register-indirect jumps, as required by the specification, causing PC misalignment and fetch errors (S2). The Vortex developers noted that this bug is triggerable only by

hand-written assembly. The compiler aligns function-entry addresses, so normal kernels remain unaffected. In addition, the OpenGPGPU-Spike zero-extends boxed 32-bit floats into 64-bit registers (P1), violating Spike’s required sign-extension and causing incorrect signed comparisons when reinterpreted as integers. This bug can misdirect branches.

5.7 Security Analysis

In this section, we assess the security impact of the discovered bugs.

Out-of-Range Memory Access. Private memory address miscalculation caused by V1 can trigger out-of-bounds accesses, enabling memory corruption. We assume an attacker can launch and execute a custom kernel on the target GPU. Under this threat model, V1 can trigger out-of-bounds accesses to private memory, enabling reads or modifications of sensitive data.

Data-Flow Integrity. Register corruption can taint critical computations, enabling malicious calculations or bypassing safety checks (V4). For example, an attacker can craft malicious code that invokes `regext` to set the register-index extension to 32 and then writes to register $32 + R$. Under ISA-level formal checks, the program is expected not to modify a critical register R . However, due to V4, the floating-point result can be erroneously written into R at runtime, thereby hijacking the data flow and affecting subsequent computations that depend on R .

Control-Flow Hijack. In ROP-style attacks, attackers seek gadgets ending in `ret` (i.e., `jalr`). If `jalr` leaves the LSB uncleared, targets can point into the middle of instructions, greatly expanding gadget availability and power. Fortunately, bug S2 exists only in the ISS. The hardware automatically truncates the low bits of the PC. Vector compare bug (V5) that deviates from the ISS may misdirect control-flow branches, causing them to follow paths unintended by programmers or the LLVM compiler.

Denial of Service. An adversarial co-tenant can execute a crafted instruction sequence to trigger an instruction-cache deadlock, hanging the GPU hardware (currently observed in our RTL simulator) and causing a denial-of-service condition (V12). Moreover, incorrect MISA configuration may cause software to mis-detect ISA extensions and disable supported features (V6).

Cryptographic Impact. Cryptographic libraries can be potentially impacted by arithmetic errors (V10), because many implementations rely on multi-precision arithmetic. Multiply-high instructions are commonly used to obtain the upper limb of a product in big-integer multiplication and modular reduction (e.g., Montgomery/Barrett-style routines). With V10, returning the low 32 bits instead of the high word can corrupt carry propagation and reduction steps, yielding incorrect residues or intermediate values.

6 Discussion

This section discusses the limitations of FuzzGPU and potential directions for improvement.

Open-Source GPU Targets. Open-source RISC-V hardware has facilitated processor research and has recently enabled explorations in high-performance GPU designs. We evaluated FuzzGPU on Vortex [41] and OpenGPGPU [27, 28], two full-stack open-source GPGPUs that support RTL validation, FPGA prototyping, and commodity GPU programming models. OpenGPGPU is the largest open-source GPU we identified by coverable lines. It has 26.7K coverable lines according to Verilator’s metric, while other available designs are much smaller (e.g., MIAOW, Nyuzi, Vortex, and TinyGPU). The targets are also practical: Vortex supports OpenCL, OpenGL, and FPGA deployment [41]. It has been integrated into a METASAT safety-critical RISC-V SoC platform [16]. OpenGPGPU also provides OpenCL support.

Side Effects of Focusing on Completion Rates. FuzzGPU improves testing efficiency by generating valid tests with high completion rates. However, this design choice excludes bugs that manifest only through fault-handling paths, such as reserved opcodes, illegal CSRs, malformed SIMT state, or unaligned memory accesses. Future work could generate invalid programs with illegal instruction encodings, invalid memory addresses, malformed SIMT control flow, and illegal register accesses to explore these paths.

Scalability to Larger Designs. We evaluated FuzzGPU on two open-source GPUs, which are small relative to commercial designs. Scaling to larger GPUs is bottlenecked by the RTL emulation time identified in Figure 14. Fortunately, hardware-accelerated emulators for large-scale RTL, such as Cadence Palladium and Synopsys ZeBu, may further improve throughput by offloading from software simulators.

Porting to Other ISAs. Although FuzzGPU follows a unified GPU execution model, its current implementation targets two specific ISAs: Vortex and OpenGPGPU. The two ISAs have different semantics. Adapting FuzzGPU to other publicly released ISAs, such as AMD Southern Islands, would necessitate modifications to the ISS, differential testing harness, and instruction generator. We anticipate that porting to proprietary ISA ecosystems, such as NVIDIA PTX and SASS, will be more challenging due to undocumented features and increased instruction set complexity.

Fuzzing Closed-Source GPUs. Many commercial GPU vendors offer soft IP cores that are source-encrypted but permit RTL simulation. Applying FuzzGPU to these closed-source cores is technically feasible but requires vendor cooperation to expose the necessary DPI-C interfaces. We have designed our DPI-C instrumentation to be minimal. For instance, implementing such instrumentation in OpenGPGPU required only about 90 lines of Chisel code. However, instrumenting encrypted sources is impossible without vendor assistance.

Limited Baselines. To the best of our knowledge, no publicly

available GPU hardware fuzzer exists for comparison. We therefore adapted Cascade [39] and DiveFuzz [10], which generate instruction-level data flows and control flows. We also considered SIMD-oriented fuzzers. RVISmith [11], however, fuzzes compilers via SIMD data-flow sequences in strip-mined loops, rather than the richer control-flow instructions needed for SIMT-aware generation. Future work could adapt RVISmith-style SIMD generation with SIMT-aware control, synchronization, and RTL-level differential checking to broaden the set of GPU fuzzers available for comparison.

7 Related Work

GPU Hardware Assessment. Prior works on GPU reliability fundamentally fall under fault injection and stress campaigns [8, 35, 36, 38, 51]. They typically inject or stress known fault modes and then measure resulting error rates or mitigation effectiveness, rather than systematically searching for previously unknown functional bugs. Accordingly, such studies are outside the scope of this paper. Concurrent with our work, Ventus developed GVM, a UVM-like verification framework for OpenGPGPU [27, 28]. However, it neither fuzzes GPUs nor validates memory consistency models.

GPU Software Fuzzing. GPU software fuzzers target drivers and runtime libraries. Moneta [18] performs *ex vivo* driver fuzzing without physical hardware, while Fuzz4CUDA [54] targets CUDA libraries and leverages runtime analysis to aid device-side coverage collection.

Processor Fuzzing. Prior processor fuzzers predominantly target CPUs rather than GPUs [3–5, 7, 10, 12, 15, 19, 20, 23, 24, 39, 40, 42, 45–47]. Based on the bug-detection oracle they employ, CPU fuzzers broadly fall into four categories: (i) assertion-based, (ii) Golden Reference Model (GRM)-based, (iii) equivalence-based, and (iv) termination-state-based.

Assertion-based fuzzers (e.g., RFuzz [24], DirectFuzz [4], Hardware Fuzzing [42]) use user-provided assertions (e.g., in SystemVerilog) as oracles, and often rely on coverage-guided mutation to generate test stimuli. GRM-based methods compare hardware states against a golden reference model. Several fuzzers adopt instruction-level differential testing as an oracle [5, 7, 10, 20, 45, 47], while RISCover [40] compares multiple closed-source RISC-V cores to expose inconsistencies. Equivalence-based approaches (e.g., EPEX [22]) generate semantically equivalent (or closely related) program variants to explore different data or control paths. Fuzzilicon [25] further extends this technique toward microcode-level vulnerabilities. Termination-state-based approaches detect bugs via non-termination. Cascade [39] constructs entangled control- and data-flow patterns and flags bugs by observing nonterminating executions.

These fuzzers were originally designed for CPUs and do not directly transfer to GPUs, because GPUs' SIMT execution model and parallel thread organization differ substantially from those of CPUs. To the best of our knowledge, no publicly

available GPU hardware fuzzer exists that targets RTL designs and supports SIMT-aware test generation and checking.

8 Conclusion

This paper presents FuzzGPU, the first RTL-level fuzzing framework for GPU hardware. FuzzGPU addresses GPU-specific architectural challenges by generating valid Single-Instruction, Multiple-Thread (SIMT) programs with diverse data and control flows that conform to the execution model, enabling effective GPU fuzzing where state-of-the-art processor fuzzers fall short. We further design and implement a GPU-oriented differential testing mechanism that mitigates microarchitectural non-determinism and enables instruction-level vulnerability detection and localization. Experimental evaluations on two open-source GPUs, Vortex and OpenGPGPU, show that FuzzGPU uncovers 20 previously unknown bugs, 10 of which have been assigned CVE identifiers.

9 Acknowledgement

This work was partially supported by the Beijing Key Laboratory of Network Security Protection Technology (Grant No. 2022YFB3103900) and E5YY07111K4. We gratefully acknowledge the constructive comments and helpful feedback provided by the anonymous reviewers.

Ethical Considerations

This section analyzes the ethical implications of FuzzGPU, including potential impacts on stakeholders, mitigations we implemented, and our rationale for publishing the work.

Stakeholder Analysis. We outline several major stakeholder groups that could be affected by the outcomes of this research.

- *Maintainers of Vortex, OpenGPGPU, Vortex-Simx, and OpenGPGPU-Spike.* These stakeholders bore the most immediate burden from our work: reproducing reports, triaging traces, prioritizing fixes, reviewing patches, and correcting simulator behavior. They also benefit directly from the bug reports and patches. We tried to reduce that burden by providing traces, proof-of-concept inputs, and patches so they could validate and remediate the problems more quickly, rather than forcing downstream adopters to discover and debug them independently.

- *Downstream developers and integrators.* Those who reuse these open-source GPU or simulator components may need to reassess their systems in light of memory safety errors, arithmetic errors, invalid write-back, control-flow mistakes, and hangs. Publication creates review and patching costs, but it also helps them avoid silently inheriting flawed components.

- *Users of systems or research platforms built on such designs.* These users may bear the downstream consequences of flawed GPU designs. If related designs are deployed, bugs of

the kind we found could affect system integrity, availability, or memory safety. The benefit to this group is earlier identification and remediation of such flaws before the designs are adopted more widely.

- *Researchers and the broader hardware-security community.* The main benefit to this group is a concrete methodology for SIMT-aware GPU hardware fuzzing, plus artifacts and baselines for future verification. The main burden is that others may spend time porting the verification to other GPUs. We judged this tradeoff acceptable because the work fills a real gap without human-subject interaction, or privacy-invasive data collection.

- *Potential malicious actors.* An adversary might adapt our methodology to analyze related open-source or derivative GPU designs. This is a dual-use risk. The expertise threshold is high, but technically capable actors could still benefit from publication.

- *Disclosure coordinators and CVE-assignment institutions.* These stakeholders incur coordination work, but also provide accountability and clearer communication. CVE IDs were assigned by a CNA following our request through MITRE.

Mitigations. We mitigated harm in three main ways. First, we limited the research to isolated simulation of open-source targets rather than probing deployed systems, because simulation sufficed for study verification while avoiding risks to uninvolved parties, including service disruption, operational costs, and unintended live-state changes.

Second, we responsibly disclosed the vulnerabilities to the affected maintainers and shared traces and proof-of-concept inputs to support validation and remediation, because disclosure-first starts remediation before broader dissemination and is less harmful than leaving maintainers or downstream users to encounter unresolved flaws without context. As of submission, the maintainers patched three vulnerabilities and scheduled an additional fix. We contributed two further patches. The remaining vulnerabilities are under evaluation. Ten CVE IDs were assigned.

Third, we frame the paper around defensive methodology and verification lessons rather than exploitation guidance.

Decision. We weighed the publication decision as a balance of benefits and costs. On the benefit side, the work was confined to local, isolated RTL simulation and ISA-level reference models and did not target public infrastructure, which reduced direct harm while preserving the main scientific benefit: finding flaws in GPU designs before they are adopted more widely. Publication also helps maintainers and future researchers scrutinize, reproduce, and build on the verification methods and bug classes presented here. On the cost side, the main ethical risk is dual use. Publishing a GPU-verification methodology with concrete bug classes may lower the barrier for others to study related open-source or derivative designs.

We considered whether not publishing the work, or withholding vulnerability details, would be preferable. That might reduce some misuse risk, but it would also weaken scien-

tificity, reproducibility, and follow-on defensive work. We therefore concluded that the expected defensive benefits are greater than the foreseeable harms.

Open Science

To support the reproducibility of our research, we provide the complete implementation of FuzzGPU, including its SIMT program generator, the differential testing harness, and the hardware configurations for Vortex and OpenGPGPU. We also provide fuzzing seed parameters and experimental scripts to reproduce the fuzzing results. We provide the GPU-ported versions of Cascade and DiveFuzz for baseline evaluation. The artifacts are available on Zenodo (<https://doi.org/10.5281/zenodo.20321391>).

References

- [1] Jade Alglave, Luc Maranget, Susmit Sarkar, and Peter Sewell. Litmus: Running tests against hardware. In Parosh Aziz Abdulla and K. Rustan M. Leino, editors, *Tools and Algorithms for the Construction and Analysis of Systems*, pages 41–44, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [2] Nils Bars, Moritz Schloegel, Tobias Scharnowski, Nico Schiller, and Thorsten Holz. Fuzztruction: using fault injection-based fuzzing to leverage implicit domain knowledge. In *Proceedings of the 32nd USENIX Conference on Security Symposium*, SEC '23, USA, 2023. USENIX Association.
- [3] Pallavi Borkar, Chen Chen, Mohamadreza Rostami, Nikhilesh Singh, Rahul Kande, Ahmad-Reza Sadeghi, Chester Rebeiro, and Jeyavijayan (JV) Rajendran. Whisperfuzz: white-box fuzzing for detecting and locating timing vulnerabilities in processors. In *Proceedings of the 33rd USENIX Conference on Security Symposium*, SEC '24, USA, 2024. USENIX Association.
- [4] Sadullah Canakci, Leila Delshadtehrani, Furkan Eris, Michael Bedford Taylor, Manuel Egele, and Ajay Joshi. Directfuzz: Automated test generation for rtl designs using directed graybox fuzzing. In *2021 58th ACM/IEEE Design Automation Conference (DAC)*, pages 529–534, 2021. doi:10.1109/DAC18074.2021.9586289.
- [5] Sadullah Canakci, Chathura Rajapaksha, Leila Delshadtehrani, Anoop Nataraja, Michael Bedford Taylor, Manuel Egele, and Ajay Joshi. Processorfuzz: Processor fuzzing with control and status registers guidance. In *2023 IEEE International Symposium on Hardware Oriented Security and Trust (HOST)*, pages 1–12, 2023. doi:10.1109/HOST55118.2023.10133714.

- [6] Katharina Ceesay-Seitz, Flavien Solt, and Kaveh Razavi. μ CFI: Formal verification of microarchitectural control-flow integrity. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, CCS '24, page 213–227, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690344.
- [7] Chen Chen, Rahul Kande, Nathan Nguyen, Flemming Andersen, Aakash Tyagi, Ahmad-Reza Sadeghi, and Jeyavijayan Rajendran. HyPFuzz: Formal-Assisted processor fuzzing. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1361–1378, Anaheim, CA, August 2023. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/chen-chen>.
- [8] Dimitris Gizopoulos, George Papadimitriou, and Odysseas Chatzopoulos. Estimating the failures and silent errors rates of cpus across isas and microarchitectures. In *2023 IEEE International Test Conference (ITC)*, pages 377–382, 2023. doi:10.1109/ITC51656.2023.00056.
- [9] Kevin Immanuel Gubbi, Mohammadnavid Tarighat, Arvind Sudarshan, Inderpreet Kaur, Pavan Dheeraj Kota, Avesta Sasan, and Houman Homayoun. State of hardware fuzzing: Current methods and the potential of machine learning and large language models. In *2025 26th International Symposium on Quality Electronic Design (ISQED)*, pages 1–7, 2025. doi:10.1109/ISQED65160.2025.11014308.
- [10] Zihui Guo, Miaomiao Yuan, Yanqi Yang, Liwei Chen, Gang Shi, and Dan Meng. Divefuzz: Enhancing cpu fuzzing via diverse instruction construction. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 1964–1978, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3719027.3765167.
- [11] Yibo He, Cunjian Huang, Xianmiao Qu, Hongdeng Chen, Wei Yang, and Tao Xie. Rvsmith: Fuzzing compilers for rvv intrinsics. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, page 768–782, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3719027.3744790.
- [12] Muhammad Monir Hossain, Arash Vafaei, Kimia Zamiri Azar, Fahim Rahman, Farimah Farahmandi, and Mark Tehranipoor. Socfuzzer: Soc vulnerability detection using cost function enabled fuzz testing. In *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pages 1–6, 2023. doi:10.23919/DATE56975.2023.10137024.
- [13] Heqing Huang, Anshunkang Zhou, Mathias Payer, and Charles Zhang. Everything is Good for Something: Counterexample-Guided Directed Fuzzing via Likely Invariant Inference. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1956–1973, Los Alamitos, CA, USA, May 2024. IEEE Computer Society. URL: <https://doi.ieeecomputersociety.org/10.1109/SP54263.2024.00142>, doi:10.1109/SP54263.2024.00142.
- [14] Rodrigo Huerta, Mojtaba Abaie Shoushtary, José-Lorenzo Cruz, and Antonio Gonzalez. Dissecting and modeling the architecture of modern gpu cores. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, MICRO '25, page 369–384, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3725843.3756041.
- [15] Jaewon Hur, Suhwan Song, Dongup Kwon, Eunjin Baek, Jangwoo Kim, and Byoungyoung Lee. Difuzzrtl: Differential fuzz testing to find cpu bugs. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1286–1303, 2021. doi:10.1109/SP40001.2021.00103.
- [16] Marc Soë i Bonet, Jannis Wolf, and Leonidas Kosmidis. A RISC-V multicore and GPU SoC platform with a qualifiable software stack for safety critical systems, 2025. URL: <https://arxiv.org/abs/2502.21027>, arXiv:2502.21027.
- [17] Yuancheng Jiang, Chuqi Zhang, Bonan Ruan, Jiahao Liu, Manuel Rigger, Roland H. C. Yap, and Zhenkai Liang. Fuzzing the php interpreter via dataflow fusion. In *Proceedings of the 34th USENIX Conference on Security Symposium*, SEC '25, USA, 2025. USENIX Association.
- [18] Joonkyo Jung, Jisoo Jang, Yongwan Jo, Jonas Vinck, Alexios Voulimeneas, Stijn Volckaert, and Dokyung Song. Moneta: Ex-vivo gpu driver fuzzing by recalling in-vivo execution states. In *NDSS*, 2025.
- [19] Nursultan Kabytkas, Tommy Thorn, Shreesha Srinath, Polychronis Kekalakis, and Jose Renau. Effective processor verification with logic fuzzer enhanced co-simulation. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, MICRO '21, page 667–678, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3466752.3480092.
- [20] Rahul Kande, Addison Crump, Garrett Persyn, Patrick Jauernig, Ahmad-Reza Sadeghi, Aakash Tyagi, and Jeyavijayan Rajendran. TheHuzz: Instruction fuzzing of processors using Golden-Reference models for finding Software-Exploitable vulnerabilities. In *31st USENIX*

Security Symposium (USENIX Security 22), pages 3219–3236, Boston, MA, August 2022. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/kande>.

- [21] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. sel4: formal verification of an os kernel. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles, SOSP '09*, page 207–220, New York, NY, USA, 2009. Association for Computing Machinery. doi:10.1145/1629575.1629596.
- [22] Lucas Klemmer and Daniel Große. Epex: Processor verification by equivalent program execution. In *Proceedings of the 2021 Great Lakes Symposium on VLSI, GLSVLSI '21*, page 33–38, New York, NY, USA, 2021. Association for Computing Machinery. doi:10.1145/3453688.3461497.
- [23] Kevin Laeuffer, Vighnesh Iyer, David Biancolin, Jonathan Bachrach, Borivoje Nikolić, and Koushik Sen. Simulator independent coverage for rtl hardware languages. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3, ASPLOS 2023*, page 606–615, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3582016.3582019.
- [24] Kevin Laeuffer, Jack Koenig, Donggyu Kim, Jonathan Bachrach, and Koushik Sen. Rfuzz: Coverage-directed fuzz testing of rtl on fpgas. In *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8, 2018. doi:10.1145/3240765.3240842.
- [25] Johannes Lenzen, Mohamadreza Rostami, Lichao Wu, and Ahmad-Reza Sadeghi. Fuzzilicon: A post-silicon microcode-guided x86 cpu fuzzer. *arXiv preprint arXiv:2512.23438*, 2025.
- [26] Reese Levine, Mingun Cho, Devon McKee, Andrew Quinn, and Tyler Sorensen. Gpuharbor: Testing gpu memory consistency at large (experience paper). In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2023*, page 779–791, New York, NY, USA, 2023. Association for Computing Machinery. doi:10.1145/3597926.3598095.
- [27] Jingzhou Li, Kexiang Yang, Chufeng Jin, Xudong Liu, Zexia Yang, Fangfei Yu, Yujie Shi, Mingyuan Ma, Li Kong, Jing Zhou, Hualin Wu, and Hu He. Ventus: A high-performance open-source gpgpu based on risc-v and its vector extension. In *2024 IEEE 42nd International Conference on Computer Design (ICCD)*, pages 276–279, 2024. doi:10.1109/ICCD63220.2024.00049.
- [28] Jingzhou Li, Fangfei Yu, Mingyuan Ma, Wei Liu, Yuhang Wang, Hualin Wu, and Hu He. Risc-v-based gpgpu with vector capabilities for high-performance computing. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 33(8):2239–2251, 2025. doi:10.1109/TVLSI.2025.3574427.
- [29] Chris S Lin, Joyce Qu, and Gururaj Saileshwar. Gpuhammer: Rowhammer attacks on gpu memories are practical. In *USENIX Security*, volume 25, 2025.
- [30] Hangtian Liu, Shuitao Gan, Chao Zhang, Zicong Gao, Hongqi Zhang, Xiangzhi Wang, and Guangming Gao. Labrador: Response guided directed fuzzing for black-box iot devices. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 1920–1938, 2024. doi:10.1109/SP54263.2024.00127.
- [31] Yuwei Liu, Junquan Deng, Xiangkun Jia, Yanhao Wang, Minghua Wang, Lin Huang, Tao Wei, and Purui Su. Promefuzz: A knowledge-driven approach to fuzzing harness generation with large language models. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS '25*, page 1559–1573, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3719027.3765222.
- [32] Daniel Lustig, Sameer Sahasrabudhe, and Olivier Giroux. A formal analysis of the NVIDIA PTX memory consistency model. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS '19*, page 257–270, New York, NY, USA, 2019. Association for Computing Machinery. doi:10.1145/3297858.3304043.
- [33] Yatin A. Manerkar, Daniel Lustig, Margaret Martonosi, and Michael Pellauer. Rtlcheck: verifying the memory consistency of rtl designs. In *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture, MICRO-50 '17*, page 463–476, New York, NY, USA, 2017. Association for Computing Machinery. doi:10.1145/3123939.3124536.
- [34] Mahta Mayahinia and Mehdi Tahoori. Electromigration reliability analysis of sram-based register files in gpus and ai accelerators. In *2025 IEEE 43rd VLSI Test Symposium (VTS)*, pages 1–4, 2025. doi:10.1109/VTS65138.2025.11022883.

- [35] Josie E. Rodriguez Condia, Paolo Rech, Fernando Fernandes dos Santos, Luigi Carro, and Matteo Sonza Reorda. An effective method to identify microarchitectural vulnerabilities in gpus. *IEEE Transactions on Device and Materials Reliability*, 22(2):129–141, 2022. doi:10.1109/TDMR.2022.3166260.
- [36] Fernando F. dos Santos, Josie E. Rodriguez Condia, Luigi Carro, Matteo Sonza Reorda, and Paolo Rech. Revealing gpu vulnerabilities by combining register-transfer and software-level fault injection. In *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, pages 292–304, 2021. doi:10.1109/DSN48987.2021.00042.
- [37] Shidong Shen, Jinyu Liu, Weizhi Feng, Fu Song, and Zhilin Wu. Bmcfuzz: Hybrid verification of processors by synergistic integration of bound model checking and fuzzing. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9, 2025. doi:10.1109/ICCAD66269.2025.11240887.
- [38] Adit Singh, Sreejit Chakravarty, George Papadimitriou, and Dimitris Gizopoulos. Silent data errors: Sources, detection, and modeling. In *2023 IEEE 41st VLSI Test Symposium (VTS)*, pages 1–12, 2023. doi:10.1109/VTS56346.2023.10139970.
- [39] Flavien Solt, Katharina Ceesay-Seitz, and Kaveh Razavi. Cascade: CPU fuzzing via intricate program generation. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5341–5358, Philadelphia, PA, August 2024. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/solt>.
- [40] Fabian Thomas, Eric García Arribas, Lorenz Heterich, Daniel Weber, Lukas Gerlach, Ruiyi Zhang, and Michael Schwarz. Riscover: Automatic discovery of user-exploitable architectural security vulnerabilities in closed-source risc-v cpus. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security, CCS '25*, page 3326–3340, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3719027.3765141.
- [41] Blaise Tine, Krishna Praveen Yalamarthy, Fares Elsabbagh, and Kim Hyesoon. Vortex: Extending the risc-v isa for gpgpu and 3d-graphics. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 754–766, 2021.
- [42] Timothy Trippel, Kang G Shin, Alex Chernyakhovsky, Garret Kelly, Dominic Rizzo, and Matthew Hicks. Fuzzing hardware like software. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3237–3254, 2022.
- [43] Oreste Villa, Daniel Lustig, Zi Yan, Evgeny Bolotin, Yaosheng Fu, Niladri Chatterjee, Nan Jiang, and David Nellans. Need for speed: Experiences building a trustworthy system-level gpu simulator. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, pages 868–880, 2021. doi:10.1109/HPCA51647.2021.00077.
- [44] Yingchen Wang, Riccardo Paccagnella, Zhao Gang, Willy R. Vasquez, David Kohlbrenner, Hovav Shacham, and Christopher W. Fletcher. Gpu.zip: On the side-channel implications of hardware-based graphical data compression. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 3716–3734, 2024. doi:10.1109/SP54263.2024.00084.
- [45] Lichao Wu, Mohamadreza Rostami, Huimin Li, Jeyavijayan Rajendran, and Ahmad-Reza Sadeghi. GenHuzz: an efficient generative hardware fuzzer. USENIX Association, USA, 2025.
- [46] Lichao Wu, Mohamadreza Rostami, Huimin Li, and Ahmad-Reza Sadeghi. Hfl: Hardware fuzzing loop with reinforcement learning. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7, 2025. doi:10.23919/DATE64628.2025.10993080.
- [47] Jinyan Xu, Yiyuan Liu, Sirui He, Haoran Lin, Yajin Zhou, and Cong Wang. MorFuzz: Fuzzing processor via runtime instruction morphing enhanced synchronizable co-simulation. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 1307–1324, Anaheim, CA, August 2023. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity23/presentation/xu-jinyan>.
- [48] Yinan Xu, Zihao Yu, Dan Tang, Guokai Chen, Lu Chen, Lingrui Gou, Yue Jin, Qianruo Li, Xin Li, Zuojun Li, Jiawei Lin, Tong Liu, Zhigang Liu, Jiazhan Tan, Huaqiang Wang, Huizhe Wang, Kaifan Wang, Chuanqi Zhang, Fawang Zhang, Linjuan Zhang, Zifei Zhang, Yangyang Zhao, Yaoyang Zhou, Yike Zhou, Jiangrui Zou, Ye Cai, Dandan Huan, Zusong Li, Jiye Zhao, Zihao Chen, Wei He, Qiyuan Quan, Xingwu Liu, Sa Wang, Kan Shi, Ninghui Sun, and Yungang Bao. Towards developing high performance risc-v processors using agile methodology. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 1178–1199, 2022. doi:10.1109/MICRO56248.2022.00080.
- [49] Yiru Xu, Hao Sun, Jianzhong Liu, Yuheng Shen, and Yu Jiang. Saturn: Host-gadget synergistic usb driver fuzzing. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4646–4660, 2024. doi:10.1109/SP54263.2024.00051.

- [50] Kaiyuan Yang, Matthew Hicks, Qing Dong, Todd Austin, and Dennis Sylvester. A2: Analog malicious hardware. In *2016 IEEE Symposium on Security and Privacy (SP)*, pages 18–37, 2016. doi:10.1109/SP.2016.10.
- [51] Lishan Yang, George Papadimitriou, Dimitrios Sartzetakis, Adwait Jog, Evgenia Smirni, and Dimitris Gizopoulos. Probing weaknesses in gpu reliability assessment: A cross-layer approach. In *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 331–333, 2024. doi:10.1109/ISPASS61541.2024.00048.
- [52] Kunlin You, Yinan Xu, Kehan Feng, Luoshan Cai, Yaoyang Zhou, and Yungang Bao. *DiffTest-H: Toward Semantic-Aware Communication in Hardware-Accelerated Processor Verification*, page 1462–1476. Association for Computing Machinery, New York, NY, USA, 2025. URL: <https://doi.org/10.1145/3725843.3756108>.
- [53] Zhenkai Zhang, Kunbei Cai, Yanan Guo, Fan Yao, and Xing Gao. Invalidate+Compare: A Timer-Free GPU cache attack primitive. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 2101–2118, Philadelphia, PA, August 2024. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity24/presentation/zhang-zhenkai>.
- [54] Yuhao Zhou, Peng Jia, Jiayong Liu, and Ximing Fan. Fuzz4cuda: Fuzzing your nvidia gpu libraries through debug interface. *Computers & Security*, 161:104754, 2026. URL: <https://www.sciencedirect.com/science/article/pii/S0167404825004432>, doi:10.1016/j.cose.2025.104754.

A Deadlock-Free Proof of Barrier Planner

We prove that the proposed barrier planner guarantees deadlock freedom. Let $B = (b_1, b_2, \dots, b_N)$ be the global barrier sequence. For any instruction i executed by thread t , we define its rank $R(i)$ as the index of the next barrier t will encounter in B , i.e., $R(i) = \min\{k \mid b_k \in B \wedge t \in \text{scope}(b_k) \wedge b_k \text{ succeeds } i \text{ in } \text{po}_t\}$. If no such barrier exists, $R(i) = N + 1$. Since each thread generates barriers following the sorted order of B , for any two instructions i_1, i_2 in thread t , $i_1 \xrightarrow{\text{po}_t} i_2 \implies R(i_1) \leq R(i_2)$.

Consider the $g\text{-HB}$ relation. If $i_1 \xrightarrow{g\text{-HB}} i_2$ via barrier b_k , then by definition i_1 precedes b_k in some thread t_1 , and i_2 succeeds b_k in some thread t_2 . Because i_1 precedes b_k , its next barrier must be b_k or an earlier one, so $R(i_1) \leq k$. Because i_2 succeeds b_k , its next barrier must be strictly later than b_k in the sequence filtered for t_2 , so $R(i_2) > k$. Thus, $i_1 \xrightarrow{g\text{-HB}} i_2 \implies R(i_1) < R(i_2)$.

A deadlock implies a cycle in $g\text{-HB} \cup \text{po}$. However, traversing any edge in the cycle strictly increases the rank (for $g\text{-HB}$) or preserves it (for po). Since a cycle must contain at least one $g\text{-HB}$ edge (intra-warp dependencies alone cannot form a cycle), traversing the cycle would imply $\exists i_x : R(i_x) < R(i_x)$, a contradiction. Therefore, the planner is deadlock-free.

B Target Hardware Details

We evaluated the latest revisions of two open-source GPGPUs. Vortex was configured for 64-bit execution (commit 2189194), while OpenGPGPU is 32-bit only. We used the official `ventus-env` repository at commit 7e97907.

At submission, we used OpenGPGPU’s latest release (also mainline) and Vortex’s latest mainline. Additional backtesting on the Vortex v2.2 release reproduced 4 out of 5 bugs and revealed a new CSR bug, confirming bug persistence.

Each test uses the maximum number of warps supported by the DUT, except for OpenGPGPU, where we generate tests using half of the warp slots per SM to avoid an unconfirmed writeback error observed under full load. However, the RTL design remains in the default full-SM configuration.

Before the final fuzzing campaign, we patched V4, V7, P1, and S2 identified in a 30-second pilot run, to prevent generator crashes and recurring blockers from limiting exploration.

Aligned with processor differential testing practices, we considered two floating-point values comparable if their difference was within 2 Unit in the Last Place (ULP).

C Fully Random Baseline

The fully random OpenGPGPU baseline generates test cases without any feedback or enforced structure. Its low completion and coverage rates, measured under the 100-seed coverage ablation setting (§5.4), provide additional context.

Table 4: Fully random baseline for OpenGPGPU.

Method	Instruction Completion Rate	Expr. Coverage
FuzzGPU	97.3%	59%
Cascade	19.8%	20%
DiveFuzz	44.7%	21%
Fully Random	0.1%	14%

D Porting Cascade to GPUs

This appendix presents the technical details of porting the Cascade fuzzing framework to two open-source GPU architectures: Vortex and Ventus. Cascade was originally designed for scalar RISC-V processors and generates semantically valid random programs to detect implementation bugs.

We ported Cascade to two GPU designs: (i) **Ventus**, a vector-oriented GPU based on RV32GCV with GPU-specific extensions, and (ii) **Vortex**, a SIMT GPU that supports both RV32 and RV64 with custom thread-management extensions. **SIMT Execution Model Adaptation.** The fundamental challenge lies in initializing multiple warps and threads correctly. For Vortex, we inject a startup sequence at the beginning of each generated program:

Listing 1: Vortex SIMT Initialization Sequence

```
addi x1, x0, NUM_WARPS
lui x2, target_addr[31:12]
addi x2, x2, target_addr[11:0]
wspawn x1, x2
addi x1, x0, THREAD_MASK
tmc x1
```

For Ventus, we utilize the standard RVV vector length configuration:

Listing 2: Ventus RVV Initialization Sequence

```
addi x1, x0, NUM_THREADS
vsetvli x0, x1, e32, m1
```

Instruction Class Extensions. We extended Cascade’s instruction classification system to accommodate GPU-specific operations. Specifically, we introduce a *Thread Control* class that captures explicit SIMT thread management in Vortex (e.g., `wspawn`, `tmc`, `split`, `join`) and vector-length setup in Ventus (`vsetvli`, `vsetvl`). We also add a *Predication* class, realized via dedicated predicate instructions in Vortex (`pred`, `pred.n`) and via mask registers in Ventus. For inter-thread coordination, a *Synchronization* class accounts for Vortex barriers (local `vx.bar` and a global barrier) as well as Ventus’s barrier instruction. Finally, we separate compute-focused classes that are architecture-dependent: Ventus exposes a *Vector ALU* class (e.g., `vadd`, `vmul`, `vfma`), whereas Vortex includes a *Tensor Core* class via `wmma`. WMMA and barrier instructions are suppressed in single-thread configurations to prevent deadlock. Barrier number is limited by hardware.

Simulator Interface Abstraction. We implemented a Spike command-compatible GPU simulator interface to seamlessly integrate with Cascade’s program construction pipeline. To remain compatible with Cascade’s original CPU-centric algorithmic design, we use the context of the first active thread as a representative program state.

E Porting DiveFuzz to GPUs

DiveFuzz is a differential fuzzing framework for hardware verification that combines template-driven program construction with semantics-preserving mutations to generate valid test cases. While the original implementation targets scalar RISC-V CPU cores (e.g., XiangShan, NutShell, Rocket Chip), our goal is to extend differential fuzzing to GPU-class execution.

Rather than changing DiveFuzz’s core generation workflow, we extended a small set of well-isolated subsystems to reflect GPU execution models and toolchain constraints: (1) architecture templates, (2) instruction specifications, (3) register/operand handling, (4) compilation and linking, (5) differential-testing backends, and (6) mutation rules.

Instruction Template Modifications. We extended DiveFuzz’s template system with GPU-specific templates that encapsulate architecture-dependent prologue/epilogue sequences and execution assumptions. Concretely, Ventus templates account for vector execution and GPU-specific initialization requirements, whereas Vortex templates expose SIMT-oriented control primitives (e.g., thread masking, divergence management, and barriers). This preserves DiveFuzz’s original template and mutation design. We use a 1:3 ratio of template generation to mutation for each baseline testcase.

We expanded DiveFuzz’s instruction template. For Ventus, this primarily consists of RVV-based vector arithmetic and memory instructions, complemented by a small set of Ventus-specific extensions. For Vortex, we added SIMT control instructions (encoded using assembler directives) to model thread management, synchronization, and divergence. Overall, the extension is declarative: new formats and operand constraints are registered without altering the higher-level generation pipeline.

Register File Names. GPU backends introduce operand conventions that differ from scalar cores. Ventus uses a unified register file with architectural mechanisms to select integer vs. floating-point banks. Consequently, our generator normalizes register naming and inserts the required prefixing where necessary. We also added support for Ventus vector registers and their extended encodings. In contrast, Vortex largely follows conventional integer/floating-point register naming, with the main additions being SIMT-specific control operands.

Compiler Configuration. Ventus relies on an LLVM/Clang-based toolchain with an explicit target configuration (e.g., `-target=riscv32` and `-mcpu=ventus-gpgpu` for assembling, and `-nostdlib` for linking). In contrast, Vortex uses a conventional GNU toolchain (`riscv32/64-unknown-elf-as/ld`) with architecture-specific `-march` settings (`-march=rv32ifmd_zicsr_zifencei` or `-march=rv64ifmd_zicsr_zifencei`). This abstraction enables the same generator to emit assembly and produce executables across both GPU backends.

GPU-Aware Spike Engine. To maintain compatibility with DiveFuzz’s original algorithmic framework, we integrated a GPU-capable ISS backend via a lightweight pybinding adapter that provides a Spike-compatible control interface while exposing GPU-specific bootstrap parameters. We use the context of the first active thread as a proxy for the program state. The session manager selects the appropriate ISS build based on the target architecture and bit width, allowing a uniform harness to drive both Ventus and Vortex executions.